

Slurm Workload Manager: Installation & Configuration

Cluster: Virtual HPC Cluster

Slurm Version: 20.11.9

Slurm Controller: hpc-master (192.168.100.1)

Compute Nodes: compute1 (192.168.100.13), compute2 (192.168.100.14)

Authentication: MUNGE

1. What is Slurm?

Slurm (Simple Linux Utility for Resource Management) is a free, open-source, highly scalable workload manager and job scheduler designed for Linux clusters of all sizes. It is one of the most widely used job schedulers in HPC environments worldwide, used by many of the world's top supercomputers.

Slurm was originally developed at Lawrence Livermore National Laboratory and is now maintained by SchedMD. It provides three key functions in an HPC cluster:

Function	Description
Resource Management	Allocates exclusive or shared access to compute nodes for a specified duration
Job Scheduling	Provides a framework for starting, executing, and monitoring jobs on allocated nodes
Queue Management	Manages a queue of pending jobs, arbitrating contention for resources

2. Key Concepts

2.1 Slurm Controller (slurmd)

The central management daemon that runs on the master node. It monitors all resources and jobs in the cluster. If the primary controller fails, a backup controller can take over. It is responsible for accepting job submissions, scheduling jobs, and tracking cluster state.

2.2 Slurm Daemon (slurmd)

The compute node daemon that runs on each worker node. It waits for jobs assigned by `slurmctld`, executes them, and reports status back to the controller. Without `slurmd` running on a node, that node cannot accept jobs.

2.3 MUNGE Authentication

MUNGE (MUNGE Uid 'N' Gid Emporium) is the authentication system used by Slurm. It creates and validates credentials to ensure secure communication between Slurm components across nodes. All nodes must have an identical MUNGE key for authentication to work.

2.4 Partition

A partition is a logical grouping of nodes with associated limits and access controls — similar to a queue in other schedulers. Jobs are submitted to partitions. In this cluster the default partition is named `normal`.

2.5 Job

A job is a resource allocation request. It typically consists of a batch script containing the commands to execute along with resource requirements specified via `#SBATCH` directives.

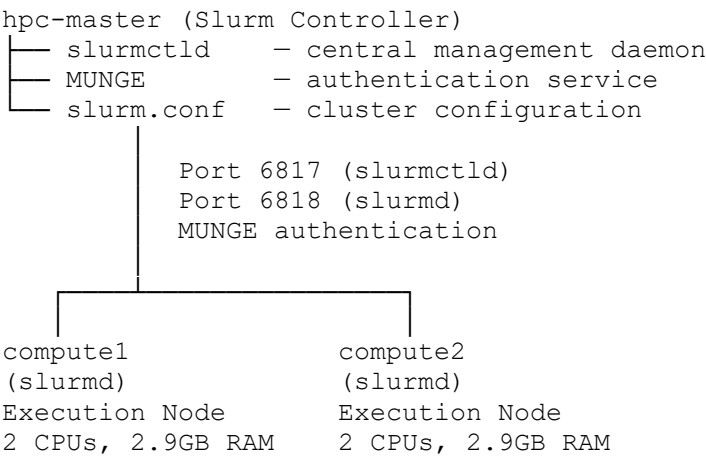
2.6 Job States

State	Code	Meaning
Pending	PD	Job waiting for resources
Running	R	Job currently executing
Completing	CG	Job finishing up
Completed	CD	Job finished successfully
Failed	F	Job failed
Cancelled	CA	Job cancelled by user or admin

2.7 TRES (Trackable RESources)

Resources that Slurm tracks and allocates — CPUs, memory, GPUs, and other consumable resources.

3. Slurm Architecture in This Cluster



Slurm Component Summary:

Component	Node	Process	Purpose
Controller	hpc-master	slurmctld	Manages jobs and resources
Worker	compute1	slurmd	Executes jobs
Worker	compute2	slurmd	Executes jobs
Auth	All nodes	munged	MUNGE authentication

4. Prerequisites

4.1 Cluster Requirements Before Slurm Installation

The following were configured before Slurm installation:

Requirement
Consistent /etc/hosts on all nodes
Passwordless SSH from master to compute nodes
Firewall disabled on all nodes
SELinux disabled on all nodes
Chrony time sync configured
NFS shared /home mounted on all nodes
Same slurm user UID on all nodes

4.2 Why Same UID Matters

Slurm runs jobs as the submitting user. If UIDs differ across nodes, jobs will run as the wrong user or fail with permission errors. The `slurm` service user must have UID 1001 on all nodes.

5. MUNGE Installation & Configuration

MUNGE must be installed and running on all nodes before Slurm. All nodes must share an identical MUNGE key.

5.1 Install EPEL Repository

MUNGE and Slurm packages are available via EPEL. Since compute nodes have no internet access, the EPEL RPM was downloaded on master and copied manually:

```
# On master (has internet)
cd /tmp
wget https://dl.fedoraproject.org/pub/epel/epel-release-latest-8.noarch.rpm
rpm -ivh /tmp/epel-release-latest-8.noarch.rpm

# Copy to compute nodes
scp /tmp/epel-release-latest-8.noarch.rpm root@compute1:/tmp/
scp /tmp/epel-release-latest-8.noarch.rpm root@compute2:/tmp/
ssh root@compute1 "rpm -ivh /tmp/epel-release-latest-8.noarch.rpm"
ssh root@compute2 "rpm -ivh /tmp/epel-release-latest-8.noarch.rpm"

# Disable EPEL on compute nodes (no internet)
ssh root@compute1 "dnf config-manager --disable epel"
ssh root@compute2 "dnf config-manager --disable epel"
```

5.2 Install MUNGE

```
# On master
dnf install munge munge-libs munge-devel -y

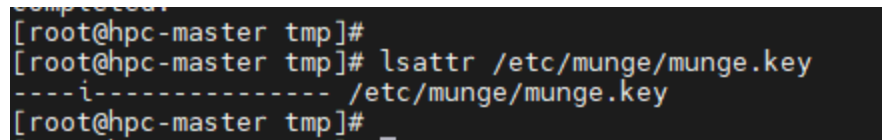
# Download RPMs and copy to compute nodes
dnf download munge munge-libs --resolve --destdir /tmp/munge-rpms
scp /tmp/munge-rpms/munge*.rpm root@compute1:/tmp/
scp /tmp/munge-rpms/munge*.rpm root@compute2:/tmp/
ssh root@compute1 "rpm -ivh /tmp/munge*.rpm --nodeps"
ssh root@compute2 "rpm -ivh /tmp/munge*.rpm --nodeps"
```

5.3 Generate MUNGE Key

```
# Generate key using /dev/urandom (faster than /dev/random)
dd if=/dev/urandom of=/etc/munge/munge.key bs=1 count=1024
chown munge:munge /etc/munge/munge.key
chmod 400 /etc/munge/munge.key

# Make immutable to prevent munged from regenerating it
chattr +i /etc/munge/munge.key

# Verify
lsattr /etc/munge/munge.key
```



```
completed
[root@hpc-master tmp]#
[root@hpc-master tmp]# lsattr /etc/munge/munge.key
----i----- /etc/munge/munge.key
[root@hpc-master tmp]#
```

Important: If `chattr +i` is not applied, munged may regenerate the key on startup causing authentication failures between nodes.

5.4 Distribute MUNGE Key to All Nodes

```
# Stop munge on compute nodes
ssh root@compute1 "systemctl stop munge"
ssh root@compute2 "systemctl stop munge"

# Copy key
scp /etc/munge/munge.key root@compute1:/etc/munge/munge.key
scp /etc/munge/munge.key root@compute2:/etc/munge/munge.key

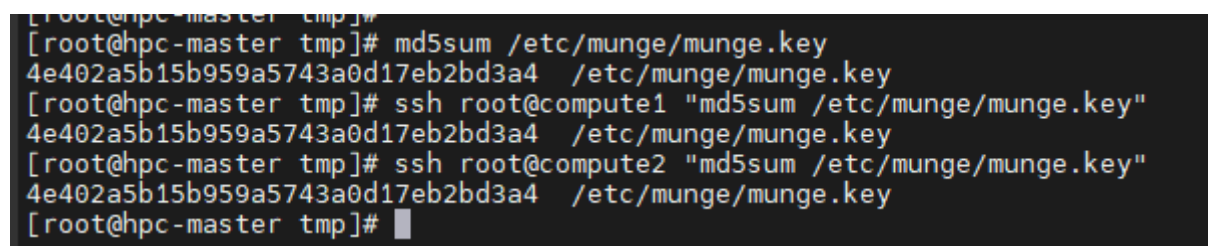
# Fix permissions and make immutable on compute nodes
ssh root@compute1 "chown munge:munge /etc/munge/munge.key && chmod 400
/etc/munge/munge.key && chattr +i /etc/munge/munge.key"
ssh root@compute2 "chown munge:munge /etc/munge/munge.key && chmod 400
/etc/munge/munge.key && chattr +i /etc/munge/munge.key"
```

5.5 Verify Keys are Identical

```
md5sum /etc/munge/munge.key
ssh root@compute1 "md5sum /etc/munge/munge.key"
ssh root@compute2 "md5sum /etc/munge/munge.key"
```

All three must show identical MD5 hash:

```
4e402a5b15b959a5743a0d17eb2bd3a4 /etc/munge/munge.key
4e402a5b15b959a5743a0d17eb2bd3a4 /etc/munge/munge.key
4e402a5b15b959a5743a0d17eb2bd3a4 /etc/munge/munge.key
```



```
[root@hpc-master tmp]#
[root@hpc-master tmp]# md5sum /etc/munge/munge.key
4e402a5b15b959a5743a0d17eb2bd3a4 /etc/munge/munge.key
[root@hpc-master tmp]# ssh root@compute1 "md5sum /etc/munge/munge.key"
4e402a5b15b959a5743a0d17eb2bd3a4 /etc/munge/munge.key
[root@hpc-master tmp]# ssh root@compute2 "md5sum /etc/munge/munge.key"
4e402a5b15b959a5743a0d17eb2bd3a4 /etc/munge/munge.key
[root@hpc-master tmp]#
```

5.6 Start MUNGE on All Nodes

```
systemctl enable --now munge
ssh root@compute1 "systemctl enable --now munge"
ssh root@compute2 "systemctl enable --now munge"
```

5.7 Verify MUNGE Authentication

```
# Test on master
munge -n | unmunge
```

```
# Test from compute nodes to master
ssh root@compute1 "munge -n | ssh root@192.168.100.1 unmunge"
ssh root@compute2 "munge -n | ssh root@192.168.100.1 unmunge"
```

Expected output:

```
STATUS:          Success (0)
ENCODE_HOST:     compute1.hpc.local (192.168.100.13)
CIPHER:          aes128 (4)
MAC:             sha256 (5)
```

```
[root@hpc-master tmp]# munge -n | unmunge
STATUS:          Success (0)
ENCODE_HOST:     hpc-master.hpc.local (192.168.100.1)
ENCODE_TIME:     2026-03-04 20:59:59 +0530 (1772638199)
DECODE_TIME:     2026-03-04 20:59:59 +0530 (1772638199)
TTL:             300
CIPHER:          aes128 (4)
MAC:             sha256 (5)
ZIP:             none (0)
UID:             root (0)
GID:             root (0)
LENGTH:         0

[root@hpc-master tmp]# ssh root@compute1 "munge -n | ssh root@192.168.100.1 unmunge"
STATUS:          Success (0)
ENCODE_HOST:     compute1.hpc.local (192.168.100.13)
ENCODE_TIME:     2026-03-04 21:00:03 +0530 (1772638203)
DECODE_TIME:     2026-03-04 21:00:06 +0530 (1772638206)
TTL:             300
CIPHER:          aes128 (4)
MAC:             sha256 (5)
ZIP:             none (0)
UID:             root (0)
GID:             root (0)
LENGTH:         0

[root@hpc-master tmp]# ssh root@compute2 "munge -n | ssh root@192.168.100.1 unmunge"
STATUS:          Success (0)
ENCODE_HOST:     compute2.hpc.local (192.168.100.14)
ENCODE_TIME:     2026-03-04 21:00:44 +0530 (1772638244)
DECODE_TIME:     2026-03-04 21:00:45 +0530 (1772638245)
TTL:             300
CIPHER:          aes128 (4)
MAC:             sha256 (5)
ZIP:             none (0)
UID:             root (0)
GID:             root (0)
```

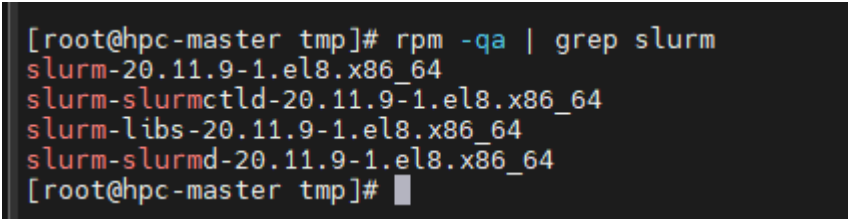
6. Slurm Installation

6.1 Install on Master

```
dnf install slurm slurm-slurmctld slurm-slurmd -y  
rpm -qa | grep slurm
```

Output:

```
slurm-20.11.9-1.el8.x86_64  
slurm-slurmctld-20.11.9-1.el8.x86_64  
slurm-libs-20.11.9-1.el8.x86_64  
slurm-slurmd-20.11.9-1.el8.x86_64
```



```
[root@hpc-master tmp]# rpm -qa | grep slurm  
slurm-20.11.9-1.el8.x86_64  
slurm-slurmctld-20.11.9-1.el8.x86_64  
slurm-libs-20.11.9-1.el8.x86_64  
slurm-slurmd-20.11.9-1.el8.x86_64  
[root@hpc-master tmp]#
```

6.2 Install on Compute Nodes

Since compute nodes have no internet, RPMs were downloaded on master and copied:

```
# Download on master  
dnf download slurm slurm-slurmd slurm-libs --resolve --destdir /tmp/slurm-  
rpms  
  
# Copy and install on compute nodes  
scp /tmp/slurm-rpms/slurm*.rpm root@compute1:/tmp/  
scp /tmp/slurm-rpms/slurm*.rpm root@compute2:/tmp/  
ssh root@compute1 "rpm -ivh /tmp/slurm*.rpm --nodeps"  
ssh root@compute2 "rpm -ivh /tmp/slurm*.rpm --nodeps"
```

7. Create Slurm User

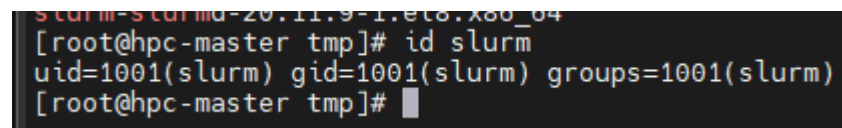
The `slurm` user must exist with identical UID on all nodes:

```
# Create on all nodes with same UID 1001
useradd -m -c "Slurm User" -d /var/lib/slurm -u 1001 -s /bin/bash slurm
ssh root@compute1 "useradd -m -c 'Slurm User' -d /var/lib/slurm -u 1001 -s /bin/bash slurm"
ssh root@compute2 "useradd -m -c 'Slurm User' -d /var/lib/slurm -u 1001 -s /bin/bash slurm"

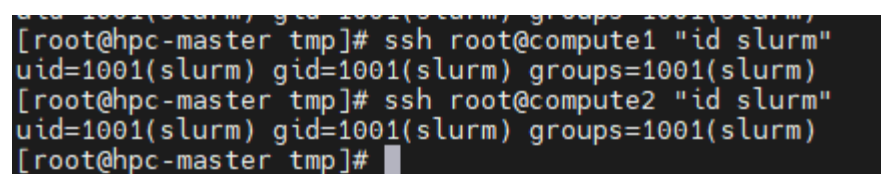
# Verify
id slurm
ssh root@compute1 "id slurm"
ssh root@compute2 "id slurm"
```

Output on all nodes:

```
uid=1001(slurm) gid=1001(slurm) groups=1001(slurm)
```



```
slurm@slurm-20.11.9-1.el8.x86_64
[root@hpc-master tmp]# id slurm
uid=1001(slurm) gid=1001(slurm) groups=1001(slurm)
[root@hpc-master tmp]#
```



```
uid=1001(slurm) gid=1001(slurm) groups=1001(slurm)
[root@hpc-master tmp]# ssh root@compute1 "id slurm"
uid=1001(slurm) gid=1001(slurm) groups=1001(slurm)
[root@hpc-master tmp]# ssh root@compute2 "id slurm"
uid=1001(slurm) gid=1001(slurm) groups=1001(slurm)
[root@hpc-master tmp]#
```

8. Create Required Directories

```
# On master
mkdir -p /var/spool/slurmctld
chown slurm:slurm /var/spool/slurmctld
chmod 755 /var/spool/slurmctld
mkdir -p /var/log/slurm
chown slurm:slurm /var/log/slurm

# On compute nodes
ssh root@compute1 "mkdir -p /var/spool/slurmd && chown slurm:slurm /var/spool/slurmd"
ssh root@compute2 "mkdir -p /var/spool/slurmd && chown slurm:slurm /var/spool/slurmd"
ssh root@compute1 "mkdir -p /var/log/slurm && chown slurm:slurm /var/log/slurm"
ssh root@compute2 "mkdir -p /var/log/slurm && chown slurm:slurm /var/log/slurm"
```

9. Configure slurm.conf

The `slurm.conf` file is the main configuration file. It must be **identical on all nodes**.

```
cat > /etc/slurm/slurm.conf << 'EOF'
# Cluster
ClusterName=hpccluster
ControlMachine=hpc-master
ControlAddr=192.168.100.1

# Authentication
AuthType=auth/munge
CryptoType=crypto/munge

# Logging
SlurmctldLogFile=/var/log/slurm/slurmctld.log
SlurmdLogFile=/var/log/slurm/slurmd.log
SlurmctldPidFile=/var/run/slurmctld.pid
SlurmdPidFile=/var/run/slurmd.pid

# Directories
StateSaveLocation=/var/spool/slurmctld
SlurmdSpoolDir=/var/spool/slurmd

# Scheduling
SchedulerType=sched/backfill
SelectType=select/cons_tres
SelectTypeParameters=CR_Core

# Timeouts
SlurmctldTimeout=300
SlurmdTimeout=300
InactiveLimit=0
MinJobAge=300
KillWait=30
Waittime=0

# Ports
SlurmctldPort=6817
SlurmdPort=6818

# Users
SlurmUser=slurm

# Job settings
JobAcctGatherType=jobacct_gather/none
AccountingStorageType=accounting_storage/none

# Nodes
NodeName=compute1 NodeAddr=192.168.100.13 CPUs=2 RealMemory=2900 Sockets=1
CoresPerSocket=2 ThreadsPerCore=1 State=UNKNOWN
NodeName=compute2 NodeAddr=192.168.100.14 CPUs=2 RealMemory=2900 Sockets=1
CoresPerSocket=2 ThreadsPerCore=1 State=UNKNOWN

# Partitions
PartitionName=normal Nodes=compute1,compute2 Default=YES MaxTime=INFINITE
State=UP
EOF
```

Key configuration parameters explained:

Parameter	Value	Meaning
ClusterName	hpccluster	Name of the cluster
ControlMachine	hpc-master	Hostname of Slurm controller
ControlAddr	192.168.100.1	IP of Slurm controller
AuthType	auth/munge	Use MUNGE for authentication
SchedulerType	sched/backfill	Backfill scheduling algorithm
SelectType	select/cons_tres	Consumable resource tracking
StateSaveLocation	/var/spool/slurmd	Where controller saves state
SlurmdSpoolDir	/var/spool/slurmd	Where slurmd saves job state
SlurmdPort	6817	Controller listening port
SlurmdPort	6818	Worker daemon port

Copy to Compute Nodes

```
scp /etc/slurm/slurm.conf root@compute1:/etc/slurm/  
scp /etc/slurm/slurm.conf root@compute2:/etc/slurm/
```

10. Start Slurm Services

10.1 Start slurmctld on Master

```
systemctl enable --now slurmctld
systemctl status slurmctld --no-pager
```

Output:

Active: active (running)

10.2 Start slurmd on Compute Nodes

```
ssh root@compute1 "systemctl enable --now slurmd"
ssh root@compute2 "systemctl enable --now slurmd"
```

```
[root@hpc-master tmp]# systemctl status slurmctld
● slurmctld.service - Slurm controller daemon
   Loaded: loaded (/usr/lib/systemd/system/slurmctld.service; enabled; vendor preset: disabled)
   Active: active (running) since Wed 2026-03-04 20:45:10 IST; 19min ago
     Main PID: 210869 (slurmctld)
        Tasks: 7
       Memory: 7.8M
      CGroup: /system.slice/slurmctld.service
              └─210869 /usr/sbin/slurmctld -D

Mar 04 20:45:10 hpc-master.hpc.local systemd[1]: Started Slurm controller daemon.
[root@hpc-master tmp]#
```

11. Verification

11.1 Check Cluster Status

sinfo

Output:

PARTITION	AVAIL	TIMELIMIT	NODES	STATE	NODELIST
normal*	up	infinite	2	idle	compute[1-2]

```
[root@hpc-master tmp]#  
[root@hpc-master tmp]# sinfo  
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST  
normal*    up      infinite    2      idle  compute[1-2]  
[root@hpc-master tmp]# █
```

11.2 Check Job Queue

squeue

Output:

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST (REASON)
-------	-----------	------	------	----	------	-------	-------------------

```
[root@hpc-master tmp]#  
[root@hpc-master tmp]# squeue  
      JOBID PARTITION      NAME      USER ST      TIME  NODES NODELIST(REASON)  
[root@hpc-master tmp]#  
[root@hpc-master tmp]#  
[root@hpc-master tmp]# █
```

12. Job Submission

12.1 Sample Job Script

```
cat > /tmp/test_job.sh << 'EOF'
#!/bin/bash
#SBATCH --job-name=TestJob
#SBATCH --partition=normal
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --time=00:01:00
#SBATCH --output=/tmp/test_job_%j.out

echo "=== Job Started ==="
echo "Running on host: $(hostname)"
echo "Date: $(date)"
echo "Job ID: $SLURM_JOB_ID"
echo "CPUs allocated: $SLURM_CPUS_ON_NODE"
echo "=== Job Finished ==="
EOF

sbatch /tmp/test_job.sh
```

12.2 Multi-Node Job

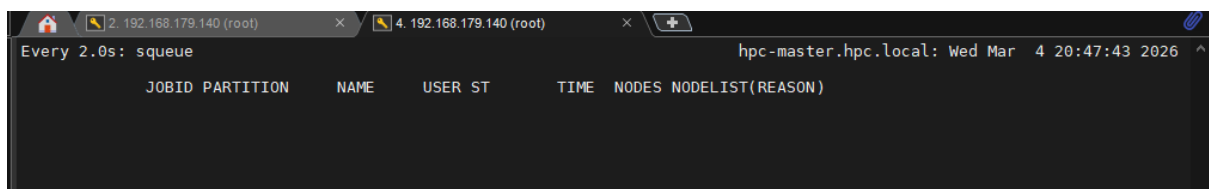
```
cat > /tmp/multi_job.sh << 'EOF'
#!/bin/bash
#SBATCH --job-name=MultiJob
#SBATCH --partition=normal
#SBATCH --ntasks=2
#SBATCH --nodes=2
#SBATCH --cpus-per-task=1
#SBATCH --time=00:01:00
#SBATCH --output=/tmp/multi_job_%j.out

echo "Job $SLURM_JOB_ID running on: $(hostname) at $(date)"
srun hostname
EOF

sbatch /tmp/multi_job.sh
```

12.3 Monitor Running Job

```
# Watch job queue
watch -n 2 squeue
```



JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
-------	-----------	------	------	----	------	-------	------------------

```
# sinfo during job execution
sinfo
```

Output during job execution:

PARTITION	AVAIL	TIMELIMIT	NODES	STATE	NODELIST
normal*	up	infinite	1	mix	compute1
normal*	up	infinite	1	idle	compute2

```
[root@hpc-master tmp]#  
[root@hpc-master tmp]# sinfo  
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST  
normal*    up      infinite     2    idle compute[1-2]  
[root@hpc-master tmp]#
```

13. SBATCH Directives

Directive	Example	Meaning
--job-name	#SBATCH --job-name=MyJob	Job name
--partition	#SBATCH --partition=normal	Queue/partition to use
--ntasks	#SBATCH --ntasks=4	Total number of tasks
--nodes	#SBATCH --nodes=2	Number of nodes
--cpus-per-task	#SBATCH --cpus-per-task=2	CPUs per task
--mem	#SBATCH --mem=4G	Memory per node
--time	#SBATCH --time=01:00:00	Max walltime
--output	#SBATCH --output=job_%j.out	Stdout file (%j = job ID)
--error	#SBATCH --error=job_%j.err	Stderr file
--mail-type	#SBATCH --mail-type=END	Email on job end

14. Important Commands

Purpose	Command
Submit a job	sbatch job.sh
Check job queue	squeue
Check cluster status	sinfo
Cancel a job	scancel <jobid>
Job details	scontrol show job <jobid>
Node details	scontrol show node compute1
Job accounting	sacct -j <jobid>
Run interactive job	srun --pty bash
Check cluster info	scontrol show config
Hold a job	scontrol hold <jobid>
Release held job	scontrol release <jobid>
Check partition info	scontrol show partition
Restart slurmctld	systemctl restart slurmctld
Restart slurmd	systemctl restart slurmd
Check slurm logs	tail -f /var/log/slurm/slurmctld.log

15. Slurm vs PBS

Feature	Slurm	OpenPBS
Job submission	sbatch	qsub
Job monitoring	squeue	qstat
Cancel job	scancel	qdel
Cluster status	sinfo	pbsnodes -a
Node daemon	slurmd	pbs_mom
Controller	slurmctld	pbs_server
Auth	MUNGE	NA
Config file	slurm.conf	pbs.conf
Directives	#SBATCH	#PBS